UNIT II: Functions and Classes & Objects in C++

Question Bank: Debugging & Pseudo-Code Formulation (Questions Only)

---

1. Syllabus & Topic Scope

- **The Main Function**: Signatures (`int main()`, `int main(int argc, char* argv[])`), return codes, execution lifecycle.
- **Function Prototyping**: Declaration vs. definition, parameter lists, type safety.
- **Call by Reference**: Syntax using references (`&`), memory alias semantics, avoiding copy overhead.
- **Return by Reference**: Returning lvalues, lvalue assignment (`func() = val`), risks of dangling references to local stack frames.
- **Inline Functions**: Syntax, compiler expansion heuristics, inline member functions, limitations (loops, recursion, static variables).
- **Default Arguments**: Placement in declarations, trailing argument rule, avoiding redeclaration conflicts.
- **Function Overloading**: Signature differentiation (types, count of parameters), resolution mechanics, ambiguity traps caused by default arguments and implicit type conversions.
- **Specifying a Class & Member Functions**: Class anatomy, access specifiers (`private`, `public`, `protected`), defining member functions inside vs. outside class using scope resolution (`::`).
- **Arrays within a Class**: Internal buffer storage, bounds containment, member methods manipulating internal arrays.
- **Static Member Functions & Static Data Members**: Class-level storage, external definition requirement, invocation via class scope (`Class::Function()`), absence of `this` pointer in static member functions.
- **Arrays of Objects**: Multi-instance allocation, dependency on default constructors, indexed manipulation.
- **Friendly Functions**: The `friend` keyword, granting non-member access to private/protected state, friend functions as bridges between classes.

---

2. Assessment Instructions & Rules ("How to Do & What to Do")

A. Instructions for Debugging Questions

When debugging functions, classes, and object interactions in C++, apply the following diagnostic methodology:

1. Analyze Lifetime & Scope:

- Inspect functions returning by reference (`Type&`). Verify whether the returned object resides in the caller's stack frame, heap memory, or static storage. If it returns a local automatic stack variable, flag it as a **Dangling Reference** causing Undefined Behavior.

2. Inspect Static Member Constraints:

- For static data members: Check if the member is only declared inside the class header or also defined and initialized at file scope outside the class. (Missing file-scope definition causes a *Linker Error*).
- For static member functions: Verify whether they attempt to access instance variables (`this->member`) or invoke non-static member functions without an explicit object instance.

3. Verify Overload Resolution & Defaults:

- Check whether default arguments cause two function signatures to become indistinguishable when called with fewer arguments (ambiguity).

4. Verify Object Array Initializability:

- If an array of objects is declared (e.g., `Class arr[10];`), verify whether the class possesses an accessible **Default Constructor** (zero arguments).

5. Inspect Friend Function Mechanics:

- Remember that a friend function is **NOT a member function**. It does not have an implicit `this` pointer, cannot be invoked via `obj.friendFunc()`, and must access private members through explicitly passed object references.

6. Required Output Format for Each Problem:

- State exact line numbers and defect classification.
- Provide an in-depth root cause analysis referencing C++ standards.
- Provide the clean, compilable corrected code snippet.
- Provide the dry run and predicted terminal output.

B. Instructions for Pseudo-Code Writing Questions

When formulating pseudo-code for Object-Oriented designs:

1. Structural Class Syntax:

- Clearly delineate class sections: `CLASS <Name>`, `PRIVATE:`, `PUBLIC:`, `ENDCLASS`.
- Distinguish between instance attributes and `STATIC` class attributes.
- Indicate parameter passing mechanisms explicitly: by value vs. by reference (`REF`).

2. Method Definitions:

- Show constructor declarations (`CONSTRUCTOR(...)`), instance methods (`METHOD ...`), and class methods (`STATIC METHOD ...`).
- When demonstrating member functions defined outside the class, use scope notation (`METHOD ClassName::MethodName()`).

3. Traceability:

- Include a dry-run trace table tracking variable and object state across method calls.

---

3. Section A: Debugging Questions (5 Problems)

Question 1: Dangling Reference in Return-by-Reference

Topic: Return by Reference, Scope, Activation Records

Problem Statement

A developer writes a utility function designed to return the scaled maximum of two integers by reference. Another function returns an element of an array passed by reference. While the array function works as intended, calling the scaled maximum function leads to segmentation faults or corrupted garbage values when accessed later in `main()`.

Buggy Code

```
#include <iostream>

// Line 4: Utility to scale and return the larger value
int& getMaximumScaled(int a, int b, int scaleFactor) {
    int maxVal = (a > b) ? a : b;
    int result = maxVal * scaleFactor;
    return result; // Warning / Undefined Behavior!
}

// Line 11: Array element modifier
int& getArrayElement(int arr[], int index) {
    return arr[index];
}

int main() {
    int x = 10, y = 20;

    // Line 19: Receiving the reference
    int &danglingRef = getMaximumScaled(x, y, 2);
    std::cout << "Scaled Max: " << danglingRef << std::endl;

    int data[3] = {100, 200, 300};
    getArrayElement(data, 1) = 999; // Lvalue assignment
    std::cout << "Updated Array[1]: " << data[1] << std::endl;

    // Line 27: Reading reference after subsequent stack calls
    std::cout << "Dangling Ref Value: " << danglingRef << std::endl;
    return 0;
}
```

Student Tasks

1. Explain what happens to the memory location of `result` on **Line 8** after `getMaximumScaled` finishes execution. Why does `danglingRef` exhibit undefined behavior on **Line 27**?

2. Why is `getArrayElement` on **Line 11** safe for return-by-reference, whereas `getMaximumScaled` is unsafe?

3. Rewrite the code using proper return semantics.

4. Show the dry-run and expected output of the corrected code.

Question 2: Static Data Member Linkage & Member Function Scoping

Topic: Static Data Members, Static Member Functions, Scope Resolution

Problem Statement

A banking software module tracks total branch reserves using a static variable in class `BankAccount`. The developer creates a static member function `displayAudit()` to inspect total reserves, but also attempts to print individual account holder information from inside the static method. The code fails with both compilation and linker errors.

Buggy Code

```
#include <iostream>
#include <string>

class BankAccount {
private:
    std::string accHolder;
    double balance;
    static double totalBankReserves; // Line 8: Static data member

public:
    BankAccount(std::string name, double initialDeposit) {
        accHolder = name;
        balance = initialDeposit;
        totalBankReserves += initialDeposit;
    }

    // Line 17: Static member function
    static void displayAudit() {
        std::cout << "Total Reserves in Vault: $" << totalBankReserves << std::endl;
        std::cout << "Account Holder: " << accHolder << std::endl; // Error!
    }
};

int main() {
    BankAccount acc1("Alice", 5000.0);
    BankAccount acc2("Bob", 3000.0);

    BankAccount::displayAudit();
    return 0;
}
```

Student Tasks

1. Identify the **Linker Error** (`undefined reference to BankAccount::totalBankReserves`) that occurs during the build process. Explain why declaring a static member variable inside a class is insufficient to allocate memory.

2. Identify the **Compilation Error** on **Line 19**. Explain why static member functions cannot access non-static member variables like `accHolder`.

3. Provide the complete corrected C++ code, including the necessary external definition of the static data member and separated display methods.

4. Show the console output of the corrected program.

Question 3: Function Overload Ambiguity via Default Arguments

Topic: Function Overloading, Default Arguments

Problem Statement

A geometric calculation library provides overloaded functions to calculate the volume of boxes and cylinders. The author provides default arguments for cuboids to allow flexible dimensions. However, invoking `computeVolume` with a single argument triggers a compilation failure due to ambiguity.

Buggy Code

```
#include <iostream>

// Overload 1: Cuboid volume with default dimensions
double computeVolume(double length, double width = 1.0, double height = 1.0) {
    return length * width * height;
}

// Overload 2: Dedicated function for cube volume
double computeVolume(double side) {
    return side * side * side;
}

// Overload 3: Cylinder volume calculation
double computeVolume(double radius, double height) {
    return 3.14159 * radius * radius * height;
}

int main() {
    // Line 19: Call with 1 argument fails compilation
    std::cout << "Volume: " << computeVolume(5.0) << std::endl;

    // Line 22: Call with 2 arguments
    std::cout << "Cylinder: " << computeVolume(3.0, 7.0) << std::endl;
    return 0;
}
```

Student Tasks

1. Explain why the compiler generates `error: call of overloaded 'computeVolume(double)' is ambiguous` on **Line 19**. Which overloaded signatures are in conflict, and why?

2. Analyze the call on **Line 22** (`computeVolume(3.0, 7.0)`). Does it also conflict with Overload 1? Explain the matching process.

3. Refactor the function signatures to eliminate ambiguity completely while retaining flexible volume calculations.

4. Provide the corrected C++ code and the resulting output.

Question 4: Array of Objects Missing Default Constructor

Topic: Arrays of Objects, Constructors, Class Specification

Problem Statement

A student records database creates an array of `Student` objects. The author defines a parameterized constructor to assign roll numbers and names during object creation. However, declaring the array `Student batch[3];` causes the compiler to reject the program.

Buggy Code

```
#include <iostream>
#include <string>

class Student {
private:
    int rollNumber;
    std::string name;

public:
    // Parameterized constructor defined by programmer
    Student(int r, std::string n) {
        rollNumber = r;
        name = n;
    }

    void display() const {
        std::cout << "Roll: " << rollNumber << ", Name: " << name << std::endl;
    }
};

int main() {
    // Line 20: Array of 3 Student objects
    Student batch[3]; // Compilation Error!

    batch[0] = Student(101, "Aarav");
    batch[1] = Student(102, "Bhavna");
    batch[2] = Student(103, "Chirag");

    for (int i = 0; i < 3; ++i) {
        batch[i].display();
    }
    return 0;
}
```

Student Tasks

1. Explain the C++ rule regarding automatic synthesis of the default constructor when a parameterized constructor is defined. Why does `Student batch[3];` fail to compile?

2. Describe two different techniques to solve this problem:

- Technique A: Supplying an explicit default constructor.
- Technique B: Initializing array elements using an array initializer list of parameterized constructors.

3. Write the corrected C++ program using Technique A.

4. Show the verified console output.

Question 5: Friend Function Declaration & Scope Resolution Flaws

Topic: Friendly Functions, Forward Declarations, Member vs. Non-Member

Problem Statement

Two classes `Engine` and `TelemetryAuditor` interact in a vehicle diagnostic system. The developer attempts to create a non-member friend function `inspectDiagnostics` to inspect private variables from both classes. The code fails to compile due to missing forward declarations, unqualified member accesses, and illegal member invocation syntax.

Buggy Code

```
#include <iostream>

// Line 4: Incomplete forward declarations
class Engine;

class Engine {
private:
    int temperature;
    int rpm;

public:
    Engine(int t, int r) : temperature(t), rpm(r) {}

    // Declaring friend function
    friend void inspectDiagnostics(Engine e, TelemetryAuditor a);
};

class TelemetryAuditor {
private:
    int alertThreshold;

public:
    TelemetryAuditor(int threshold) : alertThreshold(threshold) {}

    friend void inspectDiagnostics(Engine e, TelemetryAuditor a);
};

// Line 27: Friend function implementation
void inspectDiagnostics(Engine e, TelemetryAuditor a) {
    // Line 29: Unqualified member access error
    if (e.temperature > alertThreshold) {
        std::cout << "Warning: Engine overheat!" << std::endl;
    } else {
        std::cout << "Engine operating nominally at " << e.rpm << " RPM." << std::endl;
    }
}

int main() {
    Engine v8(105, 3200);
    TelemetryAuditor audit(100);

    // Line 41: Invoking friend function as a member function
    v8.inspectDiagnostics(v8, audit);
    return 0;
}
```

Student Tasks

1. Explain why `alertThreshold` on **Line 29** cannot be accessed directly without object qualification inside `inspectDiagnostics`. Does a friend function possess a `this` pointer?

2. Explain why `v8.inspectDiagnostics(v8, audit)` on **Line 41** fails compilation. What is the correct syntax for invoking a friend function?

3. Identify why a forward declaration of `TelemetryAuditor` is required before `class Engine` is declared.

4. Provide the fully corrected, compilable code snippet.

---

4. Section B: Pseudo-Code Writing Questions (5 Problems)

Question 1: Corporate Vault Management with Static Data & Functions

Topic: Static Data Members, Static Member Functions, Encapsulation

Scenario

Design the pseudo-code for a banking branch reserve management system. The system models individual bank branches and maintains consolidated enterprise reserves at the corporate level.

Functional Requirements

1. Class `BranchOffice`:

- **Private Instance Attributes**: `branchID` (STRING), `branchName` (STRING), `currentBalance` (REAL).
- **Private Static Attributes**: `totalVaultReserves` (REAL, initialized to 0.0), `totalBranchesActive` (INTEGER, initialized to 0).

2. Instance Methods:

- `Constructor(id, name, openingDeposit)`: Sets instance fields, increments `totalVaultReserves` by `openingDeposit`, and increments `totalBranchesActive` by 1.
- `Deposit(amount)`: Adds funds to both `currentBalance` and `totalVaultReserves`.
- `Withdraw(amount) RETURNS BOOLEAN`: Checks if sufficient branch funds exist. If valid, deducts from both `currentBalance` and `totalVaultReserves`, returning `TRUE`; otherwise returns `FALSE`.

3. Static Methods:

- `GetVaultSummary()`: Outputs total branches active and aggregate reserves across all branches.
- `GetGlobalLiquidityRatio(targetReserve AS REAL) RETURNS REAL`: Returns $\frac{\text{totalVaultReserves}}{\text{targetReserve}}$.

Student Tasks

1. Write the formal structured pseudo-code defining the `BranchOffice` class and an executable driver routine `Main`.

2. Construct a trace table showing initial state, transactions on two distinct branches (`Branch1` depositing \$150k, `Branch2` withdrawing \$50k), and the resulting static vault reserve values.

Question 2: Matrix Mathematical Engine with Arrays in Class & Friend Dot-Product

Topic: Arrays within a Class, Scope Resolution `::`, Friend Functions

Scenario

Design a mathematical class `Matrix2D` that encapsulates a 3×3 grid of real numbers. Define member methods outside the class using scope resolution syntax. Implement an external `FRIEND FUNCTION` named `ComputeCrossTrace` that calculates the scalar dot product of the primary diagonals of two distinct matrix objects.

Functional Requirements

1. Class `Matrix2D`:

- **Private Attributes**: `grid` (ARRAY[3][3] OF REAL).
- **Public Declarations**:
  - `Initialize()`: Fills matrix with zeros.
  - `SetElement(row, col, value)`: Stores value with $0 \le \text{row}, \text{col} < 3$ bounds validation.
  - `GetElement(row, col) RETURNS REAL`: Retrieves value with bounds validation.
  - `Display()`: Prints matrix in 3×3 grid format.
  - `FRIEND FUNCTION ComputeCrossTrace(Matrix2D A, Matrix2D B) RETURNS REAL`.

2. Implementation Outside Class:

- Define all member methods outside the class using the scope resolution operator `Matrix2D::MethodName()`.

3. Friend Function Logic:

- Directly access private `grid` arrays of both objects `A` and `B` without getters.
- Compute $\sum_{i=0}^{2} (A.\text{grid}[i][i] \times B.\text{grid}[i][i])$.

Student Tasks

1. Write the structured pseudo-code for the class definition, outside member function implementations, and the friend function.

2. Trace the friend function calculation for two sample matrices with diagonal elements $A_{\text{diag}} = [2, 4, 6]$ and $B_{\text{diag}} = [1, 3, 5]$.

Question 3: Student Academic Roster via Array of Objects

Topic: Arrays of Objects, Member Functions, Sorting Objects

Scenario

Develop an academic grade management system using an **Array of Objects**. Each `StudentRecord` object encapsulates student information, subject scores, and grade computation.

Functional Requirements

1. Class `StudentRecord`:

- **Private Attributes**: `rollNo` (INTEGER), `studentName` (STRING), `marks` (ARRAY[3] OF REAL), `percentage` (REAL), `grade` (CHARACTER).
- **Methods**:
  - Default constructor: Sets fields to default/blank values.
  - `ReadData(r, name, m1, m2, m3)`: Populates fields and calls internal private method `CalculateResult()`.
  - `CalculateResult()`: Computes percentage as average of 3 subjects and assigns grade:

- $\ge 85\%$: 'A' | $\ge 70\%$: 'B' | $\ge 50\%$: 'C' | $< 50\%$: 'F'.
- `GetPercentage() RETURNS REAL`: Returns calculated percentage.
- `Display()`: Prints student summary.

2. Driver Program:

- Instantiate an array of 3 `StudentRecord` objects.
- Populate with sample student records.
- Sort the array of objects in descending order of percentage using Bubble Sort.
- Display the sorted rank list.

Student Tasks

1. Write the structured pseudo-code for the `StudentRecord` class and sorting driver.

2. Provide a dry-run trace table showing the sorting passes and object swaps based on percentage.

Question 4: Flexible Logger with Overloaded Functions & Inline Helpers

Topic: Function Overloading, Default Arguments, Inline Functions

Scenario

Create an industrial embedded logging engine featuring **Function Overloading**, **Default Arguments**, and **Inline Helper Functions** for high-throughput diagnostic output.

Functional Requirements

1. Inline Helper Function:

- `INLINE FUNCTION FormatPrefix(prefixTag AS STRING) RETURNS STRING`: Returns formatted tag `"[TAG] "`.

2. Overloaded Logging Functions:

- **Overload 1**: `LogEvent(message AS STRING)`: Formats and logs general informational message with default prefix `"INFO"`.
- **Overload 2**: `LogEvent(message AS STRING, severityLevel AS INTEGER = 1, timestamp AS STRING = "00:00:00")`:
  - Uses default arguments for severity and timestamp.
  - Translates severity integer (1: "LOW", 2: "MEDIUM", 3: "CRITICAL").
  - Formats output: `"[SEVERITY] @ TIMESTAMP : MESSAGE"`.
- **Overload 3**: `LogEvent(errorCode AS INTEGER, systemModule AS STRING)`:
  - Outputs hardware fault diagnostic: `"[HARDWARE_FAULT] Module: SYSTEM_MODULE -> Error Code: 0xHEX"`.

Student Tasks

1. Write the structured pseudo-code showing all function signatures and inline definitions.

2. Demonstrate how client code calls `LogEvent` with 1, 2, and 3 arguments, and trace how default arguments are resolved.

Question 5: Hardware Register Bank with Return-by-Reference

Topic: Return by Reference, Arrays within a Class, Lvalue Semantics

Scenario

Write pseudo-code for a microcontroller hardware buffer class `RegisterBank` that allows client code to read and directly overwrite internal hardware register bytes using lvalue assignment through a member function returning by reference.

Functional Requirements

1. Class `RegisterBank`:

- **Private Attributes**: `registers` (ARRAY[8] OF INTEGER), `errorSink` (INTEGER).
- **Constructor**: Initializes all 8 registers to 0 and `errorSink` to -1.
- **Method `GetRegister(index AS INTEGER) RETURNS REF INTEGER`**:
  - Bounds check: $0 \le \text{index} < 8$.
  - If valid, returns an **lvalue reference** to `registers[index]`.
  - If out-of-bounds, emits an alert and returns reference to `errorSink` to prevent illegal memory modification.
- **Method `DisplayState()`**: Dumps current values of all 8 registers in hexadecimal format.

2. Client Program:

- Use direct lvalue assignment: `bank.GetRegister(0) <- 170`, `bank.GetRegister(1) <- 255`.
- Read through reference: `readVal <- bank.GetRegister(0)`.
- Test an out-of-bounds assignment attempt: `bank.GetRegister(10) <- 999`.

Student Tasks

1. Write the formal structured pseudo-code for `RegisterBank` and driver program.

2. Explain the mechanism of return-by-reference that enables a function call to appear on the left side of an assignment operator (`func() = val`).

SECTION A – DEBUGGING QUESTIONS

Question 1: Dangling Reference in Return-by-Reference

- Line 4 / line 8 – Undefined behavior: getMaximumScaled returns a reference to result, a local automatic variable. Its lifetime ends when the function returns; the returned reference therefore dangles.
- Line 19 and line 27 – Runtime undefined behavior: danglingRef refers to an object whose lifetime has ended. Reading it may produce garbage, a crash, or apparently valid data; no exact output is guaranteed.
- getArrayElement is safe for the shown use because it returns a reference to an element of the caller-owned array, whose lifetime continues in main. The caller must still ensure the index is in range and the array outlives the reference.

```
#include <iostream>

int getMaximumScaled(int a, int b, int scaleFactor) {
    int maxVal = (a > b) ? a : b;
    return maxVal * scaleFactor; // Return by value
}

int& getArrayElement(int arr[], int index) {
    return arr[index]; // Refers to caller-owned array element
}

int main() {
    int x = 10, y = 20;

    int scaledValue = getMaximumScaled(x, y, 2);
    std::cout << "Scaled Max: " << scaledValue << std::endl;

    int data[3] = {100, 200, 300};
    getArrayElement(data, 1) = 999;
    std::cout << "Updated Array[1]: " << data[1] << std::endl;

    std::cout << "Scaled Value: " << scaledValue << std::endl;
    return 0;
}
```

Dry run: max(10, 20) = 20; scaled value = 20 × 2 = 40. The array reference selects data[1], which is changed from 200 to 999.

```
Scaled Max: 40
Updated Array[1]: 999
Scaled Value: 40
```

Question 2: Static Data Member Linkage and Member Function Scoping

- Line 8 – Linker error: the in-class declaration static double totalBankReserves; declares a class-level member but, under the traditional non-inline static-member model used here, does not provide its required out-of-class definition/storage. Define it once at namespace scope.

- Line 19 – Compilation error: static member functions have no this pointer and are not associated with one particular BankAccount object. They can access static members directly, but cannot directly access instance member accHolder.
- Correction: keep displayAudit() static for branch-wide reserves and create a non-static displayAccount() method for an individual account.

```
#include <iostream>
#include <string>

class BankAccount {
private:
    std::string accHolder;
    double balance;
    static double totalBankReserves;

public:
    BankAccount(const std::string& name, double initialDeposit)
        : accHolder(name), balance(initialDeposit) {
        totalBankReserves += initialDeposit;
    }

    static void displayAudit() {
        std::cout << "Total Reserves in Vault: $"
                  << totalBankReserves << std::endl;
    }

    void displayAccount() const {
        std::cout << "Account Holder: " << accHolder << std::endl;
        std::cout << "Account Balance: $" << balance << std::endl;
    }
};

// Required definition of the static data member.
double BankAccount::totalBankReserves = 0.0;

int main() {
    BankAccount acc1("Alice", 5000.0);
    BankAccount acc2("Bob", 3000.0);

    BankAccount::displayAudit();
    acc1.displayAccount();
    acc2.displayAccount();
    return 0;
}
```

Dry run: constructing Alice adds 5,000; constructing Bob adds 3,000. Consolidated reserves = 8,000.

```
Total Reserves in Vault: $8000
Account Holder: Alice
Account Balance: $5000
Account Holder: Bob
Account Balance: $3000
```

Question 3: Function Overload Ambiguity via Default Arguments

- Line 19 – Compilation error: computeVolume(5.0) matches both the cuboid overload (the second and third parameters have defaults) and the one-parameter cube overload. Both are viable with an exact double match, so neither is better.
- Line 22 – It is also ambiguous: computeVolume(3.0, 7.0) matches the cuboid overload using its default height and the cylinder overload with two parameters. Both arguments match exactly.
- Robust correction: use distinct descriptive function names for cube, cuboid, and cylinder calculations. This removes overload ambiguity while retaining flexible cuboid dimensions through default arguments.

```
#include <iostream>

double cubeVolume(double side) {
    return side * side * side;
}

double cuboidVolume(double length, double width = 1.0,
                    double height = 1.0) {
    return length * width * height;
}

double cylinderVolume(double radius, double height) {
    const double PI = 3.14159;
    return PI * radius * radius * height;
}

int main() {
    std::cout << "Cube: " << cubeVolume(5.0) << std::endl;
    std::cout << "Cuboid: " << cuboidVolume(5.0) << std::endl;
    std::cout << "Cylinder: " << cylinderVolume(3.0, 7.0) << std::endl;
    return 0;
}
```

Dry run: cube = $5^3 = 125$; cuboid with default width and height = $5 \times 1 \times 1 = 5$; cylinder = $3.14159 \times 3^2 \times 7 = 197.92017$.

```
Cube: 125
Cuboid: 5
Cylinder: 197.92
```

Question 4: Array of Objects Missing Default Constructor

- Line 20 – Compilation error: when a programmer declares a parameterized constructor, the compiler does not automatically generate a zero-argument default constructor. Student batch[3] requires three default-constructed Student objects, but no default constructor exists.
- Technique A: explicitly define Student() with default values. This makes Student batch[3] valid.
- Technique B: initialize every array element with a parameterized constructor, for example Student batch[3] = {Student(101, "Aarav"), Student(102, "Bhavna"), Student(103, "Chirag")};. This avoids needing a default constructor for these elements.

```
#include <iostream>
#include <string>
```

```
class Student {
private:
    int rollNumber;
    std::string name;

public:
    Student() : rollNumber(0), name("Not Assigned") {}

    Student(int r, const std::string& n)
        : rollNumber(r), name(n) {}

    void display() const {
        std::cout << "Roll: " << rollNumber
                  << ", Name: " << name << std::endl;
    }
};

int main() {
    Student batch[3];

    batch[0] = Student(101, "Aarav");
    batch[1] = Student(102, "Bhavna");
    batch[2] = Student(103, "Chirag");

    for (int i = 0; i < 3; ++i) {
        batch[i].display();
    }
    return 0;
}
```

Dry run: batch[0], batch[1], and batch[2] are assigned the three parameterized Student objects before display.

```
Roll: 101, Name: Aarav
Roll: 102, Name: Bhavna
Roll: 103, Name: Chirag
```

Question 5: Friend Function Declaration and Scope Resolution Flaws

- Line 29 – Compilation error: alertThreshold belongs to a TelemetryAuditor object. The friend function must access it as a.alertThreshold.
- A friend function is a non-member function. It has no implicit this pointer and is not called using object.memberFunction() syntax.
- A forward declaration class TelemetryAuditor; is required before Engine because Engine's friend declaration mentions TelemetryAuditor before that class is fully declared.
- Line 41 – Compilation error: call the non-member friend function directly as inspectDiagnostics(v8, audit).

```
#include <iostream>

class Engine;
class TelemetryAuditor; // Forward declaration

class Engine {
```

```
private:
    int temperature;
    int rpm;

public:
    Engine(int t, int r) : temperature(t), rpm(r) {}

    friend void inspectDiagnostics(const Engine& e,
                                   const TelemetryAuditor& a);
};

class TelemetryAuditor {
private:
    int alertThreshold;

public:
    TelemetryAuditor(int threshold) : alertThreshold(threshold) {}

    friend void inspectDiagnostics(const Engine& e,
                                   const TelemetryAuditor& a);
};

void inspectDiagnostics(const Engine& e,
                        const TelemetryAuditor& a) {
    if (e.temperature > a.alertThreshold) {
        std::cout << "Warning: Engine overheat!" << std::endl;
    } else {
        std::cout << "Engine operating nominally at "
                  << e.rpm << " RPM." << std::endl;
    }
}

int main() {
    Engine v8(105, 3200);
    TelemetryAuditor audit(100);

    inspectDiagnostics(v8, audit);
    return 0;
}
```

Dry run: engine temperature 105 is greater than the auditor threshold 100, so the warning branch executes.

```
Warning: Engine overheat!
```

SECTION B – PSEUDO-CODE WRITING QUESTIONS

The following solutions use explicit class sections, distinguish static and instance attributes, show REF where reference passing/return is required, and include object-state trace tables.

Question 1: Corporate Vault Management with Static Data and Functions

- Instance attributes store each branch's ID, name, and balance. Static attributes store consolidated reserves and active branch count shared by all BranchOffice objects.
- Deposit increases both branch balance and global reserves. Withdraw checks the branch's own balance before reducing both values.

```
CLASS BranchOffice
    PRIVATE:
        branchID: STRING
        branchName: STRING
        currentBalance: REAL
        STATIC totalVaultReserves: REAL <- 0.0
        STATIC totalBranchesActive: INTEGER <- 0

    PUBLIC:
        CONSTRUCTOR BranchOffice(id: STRING, name: STRING,
                                 openingDeposit: REAL)
            branchID <- id
            branchName <- name
            currentBalance <- openingDeposit
            totalVaultReserves <- totalVaultReserves + openingDeposit
            totalBranchesActive <- totalBranchesActive + 1
        END

        METHOD Deposit(amount: REAL)
            IF amount > 0 THEN
                currentBalance <- currentBalance + amount
                totalVaultReserves <- totalVaultReserves + amount
            ELSE
                OUTPUT "Invalid deposit"
            ENDIF
        END

        METHOD Withdraw(amount: REAL) RETURNS BOOLEAN
            IF amount > 0 AND amount <= currentBalance THEN
                currentBalance <- currentBalance - amount
                totalVaultReserves <- totalVaultReserves - amount
                RETURN TRUE
            ENDIF
            RETURN FALSE
        END

        STATIC METHOD GetVaultSummary()
            OUTPUT "Active branches: ", totalBranchesActive
            OUTPUT "Total reserves: ", totalVaultReserves
        END

        STATIC METHOD GetGlobalLiquidityRatio(targetReserve: REAL)
            IF targetReserve <= 0 THEN RETURN 0 ENDIF
            RETURN totalVaultReserves / targetReserve
```

```
        END
ENDCLASS

BEGIN Main
    DECLARE Branch1: BranchOffice
    DECLARE Branch2: BranchOffice
    Branch1 <- BranchOffice("B01", "Central", 500000)
    Branch2 <- BranchOffice("B02", "North", 300000)

    CALL Branch1.Deposit(150000)
    DECLARE success: BOOLEAN
    success <- Branch2.Withdraw(50000)

    CALL BranchOffice::GetVaultSummary()
    OUTPUT "Withdrawal successful: ", success
    OUTPUT "Liquidity ratio: ",
           BranchOffice::GetGlobalLiquidityRatio(1000000)
END
```

Trace assumptions: the question gives the transaction amounts but not opening balances. For a concrete trace, Branch1 starts with 500,000 and Branch2 with 300,000.

Stage	Branch1 balance	Branch2 balance	Static reserves	Active branches
After construction	500,000	300,000	800,000	2
Branch1 deposits 150,000	650,000	300,000	950,000	2
Branch2 withdraws 50,000	650,000	250,000	900,000	2

Final global liquidity ratio for targetReserve = 1,000,000 is 900,000 / 1,000,000 = 0.90.

Question 2: Matrix Mathematical Engine and Friend Dot-Product

- Matrix2D stores a private 3 × 3 REAL array. All member methods are declared in the class and defined outside it using Matrix2D::MethodName().
- ComputeCrossTrace is a non-member friend. It directly reads both private diagonal arrays and returns the sum of the three pairwise diagonal products.

```
CLASS Matrix2D
    PRIVATE:
        grid: ARRAY[3][3] OF REAL

    PUBLIC:
        METHOD Initialize()
        METHOD SetElement(row: INTEGER, col: INTEGER, value: REAL)
        METHOD GetElement(row: INTEGER, col: INTEGER) RETURNS REAL
        METHOD Display()
        FRIEND FUNCTION ComputeCrossTrace(REF A: Matrix2D,
                                          REF B: Matrix2D) RETURNS REAL
ENDCLASS

METHOD Matrix2D::Initialize()
    FOR row <- 0 TO 2
        FOR col <- 0 TO 2
            grid[row][col] <- 0.0
```

```
        ENDFOR
    ENDFOR
ENDMETHOD

METHOD Matrix2D::SetElement(row: INTEGER, col: INTEGER, value: REAL)
    IF row >= 0 AND row < 3 AND col >= 0 AND col < 3 THEN
        grid[row][col] <- value
    ELSE
        OUTPUT "ERROR: Matrix index out of bounds"
    ENDIF
ENDMETHOD

METHOD Matrix2D::GetElement(row: INTEGER, col: INTEGER) RETURNS REAL
    IF row >= 0 AND row < 3 AND col >= 0 AND col < 3 THEN
        RETURN grid[row][col]
    ELSE
        OUTPUT "ERROR: Matrix index out of bounds"
        RETURN 0.0
    ENDIF
ENDMETHOD

METHOD Matrix2D::Display()
    FOR row <- 0 TO 2
        FOR col <- 0 TO 2
            OUTPUT grid[row][col], " "
        ENDFOR
        OUTPUT NEWLINE
    ENDFOR
ENDMETHOD

FUNCTION ComputeCrossTrace(REF A: Matrix2D, REF B: Matrix2D) RETURNS REAL
    DECLARE total: REAL <- 0.0
    FOR i <- 0 TO 2
        total <- total + (A.grid[i][i] * B.grid[i][i])
    ENDFOR
    RETURN total
ENDFUNCTION

BEGIN Main
    DECLARE A, B: Matrix2D
    CALL A.Initialize()
    CALL B.Initialize()
    // Set diagonal entries to the values given in the question.
    CALL A.SetElement(0, 0, 2)
    CALL A.SetElement(1, 1, 4)
    CALL A.SetElement(2, 2, 6)
    CALL B.SetElement(0, 0, 1)
    CALL B.SetElement(1, 1, 3)
    CALL B.SetElement(2, 2, 5)
    OUTPUT "Cross trace: ", ComputeCrossTrace(A, B)
END
```

i	A[i][i]	B[i][i]	Product	Running total
0	2	1	$2 \times 1 = 2$	2
1	4	3	$4 \times 3 = 12$	14
2	6	5	$6 \times 5 = 30$	44

ComputeCrossTrace(A, B) = 2 + 12 + 30 = 44.

Question 3: Student Academic Roster via Array of Objects

- Each StudentRecord object encapsulates the roll number, name, three subject marks, percentage, and grade. A default constructor makes an array of three objects possible.
- Bubble Sort compares adjacent percentages and swaps complete objects when the left percentage is lower, producing descending order.

```
CLASS StudentRecord
    PRIVATE:
        rollNo: INTEGER
        studentName: STRING
        marks: ARRAY[3] OF REAL
        percentage: REAL
        grade: CHARACTER

        METHOD CalculateResult()
            percentage <- (marks[0] + marks[1] + marks[2]) / 3
            IF percentage >= 85 THEN grade <- 'A'
            ELSE IF percentage >= 70 THEN grade <- 'B'
            ELSE IF percentage >= 50 THEN grade <- 'C'
            ELSE grade <- 'F'
            ENDIF
        END

    PUBLIC:
        CONSTRUCTOR StudentRecord()
            rollNo <- 0
            studentName <- ""
            marks[0] <- 0
            marks[1] <- 0
            marks[2] <- 0
            percentage <- 0
            grade <- 'F'
        END

        METHOD ReadData(r: INTEGER, name: STRING,
                        m1: REAL, m2: REAL, m3: REAL)
            rollNo <- r
            studentName <- name
            marks[0] <- m1
            marks[1] <- m2
            marks[2] <- m3
            CALL CalculateResult()
        END

        METHOD GetPercentage() RETURNS REAL
            RETURN percentage
        END

        METHOD Display()
            OUTPUT rollNo, " ", studentName, " ",
```

```
                    percentage, "% Grade ", grade
        END
ENDCLASS

BEGIN Main
    DECLARE students: ARRAY[3] OF StudentRecord
    DECLARE i, pass: INTEGER
    DECLARE temp: StudentRecord

    students[0].ReadData(101, "Asha", 90, 86, 94)
    students[1].ReadData(102, "Bala", 75, 80, 70)
    students[2].ReadData(103, "Charan", 88, 92, 90)

    FOR pass <- 0 TO 1
        FOR i <- 0 TO 1 - pass
            IF students[i].GetPercentage() <
               students[i + 1].GetPercentage() THEN
                temp <- students[i]
                students[i] <- students[i + 1]
                students[i + 1] <- temp
            ENDIF
        ENDFOR
    ENDFOR

    OUTPUT "Rank List"
    FOR i <- 0 TO 2
        OUTPUT "Rank ", i + 1
        CALL students[i].Display()
    ENDFOR
END
```

Trace sample data: Asha = 90.00%; Bala = 75.00%; Charan = 90.00%. The percentages are based on the three supplied marks. Asha and Charan tie at 90%; the stable bubble sort preserves their original relative order.

Pass	Comparison	Swap?	Array order after comparison
Initial	—	—	Asha 90, Bala 75, Charan 90
Pass 1, i=0	90 < 75?	No	Asha 90, Bala 75, Charan 90
Pass 1, i=1	75 < 90?	Yes	Asha 90, Charan 90, Bala 75
Pass 2, i=0	90 < 90?	No	Asha 90, Charan 90, Bala 75

Final rank order: 1. Asha – 90% (A); 2. Charan – 90% (A); 3. Bala – 75% (B).

Question 4: Flexible Logger with Overloaded Functions and Inline Helpers

- The inline helper creates a tag prefix. Three distinct logging signatures support informational messages, severity/timestamp messages with defaults, and hardware fault messages.

- Overload resolution is determined by the argument types and count. A one-string call selects the one-parameter overload; a string plus integer selects the severity overload; an integer plus string selects the hardware-fault overload.

```
INLINE FUNCTION FormatPrefix(prefixTag: STRING) RETURNS STRING
    RETURN "[" + prefixTag + "] "
ENDINLINE

FUNCTION LogEvent(message: STRING)
    OUTPUT FormatPrefix("INFO") + message
ENDFUNCTION

FUNCTION LogEvent(message: STRING,
                  severityLevel: INTEGER = 1,
                  timestamp: STRING = "00:00:00")
    DECLARE severity: STRING
    IF severityLevel = 1 THEN severity <- "LOW"
    ELSE IF severityLevel = 2 THEN severity <- "MEDIUM"
    ELSE IF severityLevel = 3 THEN severity <- "CRITICAL"
    ELSE severity <- "UNKNOWN"
    ENDIF
    OUTPUT "[" + severity + "] @ " + timestamp + " : " + message
ENDFUNCTION

FUNCTION LogEvent(errorCode: INTEGER, systemModule: STRING)
    OUTPUT "[HARDWARE_FAULT] Module: " + systemModule +
           " -> Error Code: 0x" + HEX(errorCode)
ENDFUNCTION

BEGIN Main
    CALL LogEvent("System started")
    CALL LogEvent("Temperature rising", 2)
    CALL LogEvent("Motor failure", 3, "14:30:00")
    CALL LogEvent(255, "MOTOR")
END
```

Call	Selected function	Default resolution / result
LogEvent("System started")	One-string overload	Prefix defaults to INFO; output [INFO] System started.
LogEvent("Temperature rising", 2)	Severity overload	severityLevel = 2; timestamp defaults to 00:00:00.
LogEvent("Motor failure", 3, "14:30:00")	Severity overload	No defaults needed; severity = CRITICAL.
LogEvent(255, "MOTOR")	Hardware-fault overload	Integer and string arguments match this signature.

Illustrative output

```
[INFO] System started
[MEDIUM] @ 00:00:00 : Temperature rising
[CRITICAL] @ 14:30:00 : Motor failure
[HARDWARE_FAULT] Module: MOTOR -> Error Code: 0xFF
```

Question 5: Hardware Register Bank with Return-by-Reference

- GetRegister returns REF INTEGER, an alias to a persistent element of the RegisterBank object's internal array. Therefore, the returned reference can be used as an assignment target.
- For an invalid index, the method reports an alert and returns a reference to the persistent errorSink member. This avoids returning a reference to a temporary or local variable. The error sink is not one of the eight real registers.

```
CLASS RegisterBank
    PRIVATE:
        registers: ARRAY[8] OF INTEGER
        errorSink: INTEGER

    PUBLIC:
        CONSTRUCTOR RegisterBank()
            FOR i <- 0 TO 7
                registers[i] <- 0
            ENDFOR
            errorSink <- -1
        END

        METHOD GetRegister(index: INTEGER) RETURNS REF INTEGER
            IF index >= 0 AND index < 8 THEN
                RETURN REF registers[index]
            ELSE
                OUTPUT "ALERT: Register index out of bounds"
                RETURN REF errorSink
            ENDIF
        END

        METHOD DisplayState()
            FOR i <- 0 TO 7
                OUTPUT "Register[", i, "] = 0x", HEX(registers[i])
            ENDFOR
            OUTPUT "Error sink = ", errorSink
        END
ENDCLASS

BEGIN Main
    DECLARE bank: RegisterBank
    bank <- RegisterBank()

    bank.GetRegister(0) <- 170
    bank.GetRegister(1) <- 255

    DECLARE readVal: INTEGER
    readVal <- bank.GetRegister(0)
    OUTPUT "Read register 0: ", readVal

    bank.GetRegister(10) <- 999
    CALL bank.DisplayState()
END
```

Operation	Target / value	State after operation
Constructor	All eight registers initialized	[0,0,0,0,0,0,0,0]; errorSink = -1
Assignment	GetRegister(0) <- 170	registers[0] = 170 (0xAA)
Assignment	GetRegister(1) <- 255	registers[1] = 255 (0xFF)
Read	readVal <- GetRegister(0)	readVal = 170
Out-of-range write	GetRegister(10) <- 999	Alert; errorSink becomes 999; real register array unchanged

Return-by-reference mechanism: the function returns an lvalue reference to a member array element. The function call therefore denotes the original storage location rather than a copied value, allowing statements such as bank.GetRegister(0) = 170. The referenced RegisterBank object must remain alive, and the returned reference must not outlive that object.